

Course: Foundations of AI with Embeddings

Lecture 6: Sequential Models

Markov Chains · Google PageRank · Hidden Markov Models

Liubov B.T., Vladimir Kukushkin

Constructor University, 2026

Sequences are everywhere

Many real-world systems generate *sequential* data where order matters:

- **Text:** word follows word; character follows character
- **User sessions:** page $\rightarrow$ page $\rightarrow$ purchase (or exit)
- **Finance:** stock prices, trading volumes over time
- **Speech:** a stream of audio frames encoding phonemes
- **Biology:** DNA base sequences, protein residues

Goal: learn a probability distribution over sequences $X_1, X_2, \dots, X_T$.

Problem: $P(X_1, \dots, X_T) = P(X_1) \cdot P(X_2|X_1) \cdot P(X_3|X_2, X_1) \cdot \dots = P(X_1) \prod_{t=2}^T P(X_t | X_{t-1}, \dots, X_1)$ is intractable in general – the conditioning history grows without bound.

Key idea: impose a *conditional independence* assumption to tame the complexity.

1. Markov Chains

The Markov Property

A sequence $X_1, X_2, \dots$ satisfies the **(first-order) Markov property** if

$$P(X_t \mid X_{t-1}, X_{t-2}, \dots, X_1) = P(X_t \mid X_{t-1}).$$

Intuition

“The future is independent of the past, *given* the present.”

Consequence: the joint probability factorises as

$$P(X_1, \dots, X_T) = P(X_1) \prod_{t=2}^T P(X_t \mid X_{t-1}).$$

We additionally assume **time-homogeneity**: $P(X_t = j \mid X_{t-1} = i)$ does not depend on t .

Formal Definition

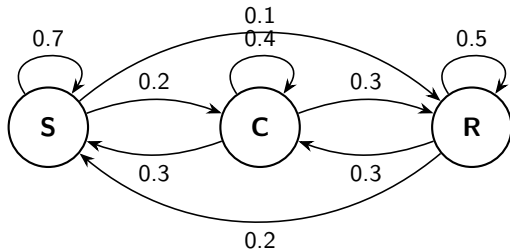
A **(homogeneous, discrete-time) Markov chain** is defined by:

- A finite **state space** $\mathcal{S} = \{s_1, \dots, s_K\}$
- A **transition matrix** $A \in \mathbb{R}^{K \times K}$, $A_{ij} = P(X_t = s_j \mid X_{t-1} = s_i)$
- An **initial distribution** $\pi_0 \in \mathbb{R}^K$, $(\pi_0)_i = P(X_1 = s_i)$

Properties of A :

- All entries non-negative: $A_{ij} \geq 0$
- **Row-stochastic:** $\sum_{j=1}^K A_{ij} = 1$ for every row i
- Entry A_{ij} is the probability of moving from state i to state j in one step

Transition Diagram



States: **S**unny, **C**loudy, **R**ainy.

Transition matrix:

$$A = \begin{pmatrix} 0.7 & 0.2 & 0.1 \\ 0.3 & 0.4 & 0.3 \\ 0.2 & 0.3 & 0.5 \end{pmatrix}$$

Row i = distribution over next states
when currently in state i .

State Distribution Evolution

Let $\pi_t \in \mathbb{R}^K$ be the **row vector** of state probabilities at time t :

$$(\pi_t)_j = P(X_t = s_j).$$

One-step update (law of total probability):

$$(\pi_t)_j = \sum_{i=1}^K (\pi_{t-1})_i A_{ij} \quad \implies \quad \pi_t = \pi_{t-1} A.$$

Multi-step propagation:

$$\pi_t = \pi_0 A^t.$$

The (i, j) entry of A^t gives the *t-step transition probability* $P(X_t = s_j \mid X_0 = s_i)$.

Example. Start in Sunny with probability 1: $\pi_0 = (1, 0, 0)$. After one step: $\pi_1 = (0.7, 0.2, 0.1)$. After two steps: $\pi_2 = \pi_1 A$.

Stationary Distribution

A distribution $\pi^* \in \mathbb{R}^K$ is **stationary** (or *invariant*) for A if

$$\pi^* = \pi^* A, \quad \sum_{j=1}^K \pi_j^* = 1, \quad \pi_j^* \geq 0.$$

π^* is a *left eigenvector* of A with eigenvalue 1: $\pi^*(A - I) = \mathbf{0}$.

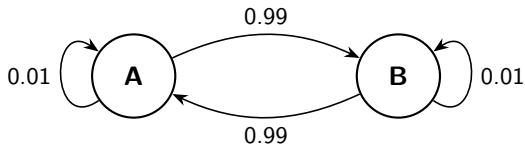
Theorem (Perron–Frobenius)

A Markov chain that is **irreducible** (any state reachable from any state) and **aperiodic** (no periodic cycles) has a **unique** stationary distribution, and $\pi_t \rightarrow \pi^*$ for any starting distribution π_0 .

Computing π^* :

- Solve the linear system $\pi^*(A - I) = \mathbf{0}$ subject to $\sum_j \pi_j^* = 1$
- *Power iteration*: run $\pi_0 A^t$ until convergence

Question: What is the stationary distribution?



Transition matrix:

$$A = \begin{pmatrix} 0.01 & 0.99 \\ 0.99 & 0.01 \end{pmatrix}$$

The chain bounces between A and B almost every step.

What is π^* ?

Answer: Stationary Distribution of the Two-State Chain

Setup: $\pi^* = (\pi_A^*, \pi_B^*)$ satisfies $\pi^* = \pi^* A$ and $\pi_A^* + \pi_B^* = 1$.

Equation for π_A^* :

$$\pi_A^* = 0.01 \pi_A^* + 0.99 \pi_B^* = 0.01 \pi_A^* + 0.99 (1 - \pi_A^*)$$

$$\pi_A^* = 0.99 - 0.98 \pi_A^* \implies 1.98 \pi_A^* = 0.99 \implies \boxed{\pi_A^* = \pi_B^* = 0.5}$$

Intuition. The chain is *symmetric*: $a_{AB} = a_{BA} = 0.99$. By symmetry, both states must be equally likely in steady state — *regardless* of how large or small the self-loop probability is.

Key takeaway. The stationary distribution depends on the *ratio* of transition rates, not their absolute magnitude.

General two-state formula

For $A = \begin{pmatrix} 1-\alpha & \alpha \\ \beta & 1-\beta \end{pmatrix}$: $\pi_A^* = \frac{\beta}{\alpha + \beta}$, $\pi_B^* = \frac{\alpha}{\alpha + \beta}$. Here $\alpha = \beta = 0.99$, so $\pi^* = (0.5, 0.5)$.

Application: Google PageRank

Naive idea: rank page j by the number of incoming links.

Problem: 100 links from spam sites $\ll$ 1 link from Wikipedia.

Recursive definition. $L = (L_{ij}) = \mathbb{1}_{j \rightarrow i}$ – a matrix of links, $c_j = \sum_i L_{ij}$ – the number of pages that page j links to. PageRank p_j is the weighted sum of ranks of pages that refer to j :

$$p_j = \sum_{i=1}^N \frac{L_{ij}}{c_i} p_i \quad \text{or in matrix form: } p = LD_c^{-1}p,$$

where $D_c = \text{diag}(c)$. So each page i distributes its rank equally among all pages it links to.

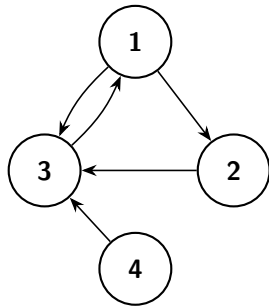
Problem: *Dangling nodes* – pages with no outgoing links.

Fix – teleportation (Page, 1998):

$$p = (1 - d)e + d \cdot LD_c^{-1}p \quad d \approx 0.85, \quad e = (1, \dots, 1)^\top$$

$$p = \left[(1 - d)ee^\top / N + dLD_c^{-1} \right] p = Ap$$

PageRank: example



Out-degrees: $c = (2, 1, 1, 1)$

$$L = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} \quad LD_c^{-1} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$
$$A = \begin{pmatrix} 0.0375 & 0.0375 & 0.8875 & 0.0375 \\ 0.4625 & 0.0375 & 0.0375 & 0.0375 \\ 0.4625 & 0.8875 & 0.0375 & 0.8875 \\ 0.0375 & 0.0375 & 0.0375 & 0.0375 \end{pmatrix}$$
$$p = (1.49, 0.78, 1.58, 0.15)$$

2. Hidden Markov Models

Observed vs. Hidden Sequences

In many settings the underlying *state* cannot be observed directly — only noisy *emissions* are visible:

Application	Hidden state	Observation
Speech recognition	Phoneme	Audio frame (MFCC vector)
POS tagging	POS tag (Noun, Verb...)	Word
Gene finding	Gene / non-gene region	DNA base (A, C, G, T)
Finance	Market regime (bull/bear)	Daily return
User modelling	User intent	Clicked item

Hidden Markov Model (HMM): extend a Markov chain by adding a layer of *probabilistic emissions* on top of the hidden states.

HMM: Definition

A **Hidden Markov Model** $\lambda = (A, B, \pi)$ consists of:

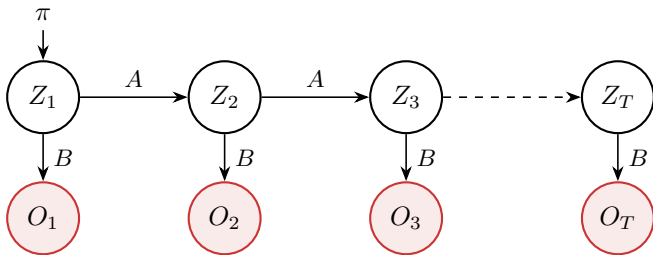
- N **hidden states** $\mathcal{Q} = \{q_1, \dots, q_N\}$
- M **observation symbols** $\mathcal{V} = \{v_1, \dots, v_M\}$
- **Transition matrix** $A \in \mathbb{R}^{N \times N}$: $a_{ij} = P(Z_t = q_j \mid Z_{t-1} = q_i)$
- **Emission matrix** $B \in \mathbb{R}^{N \times M}$: $b_{ik} = P(O_t = v_k \mid Z_t = q_i)$
- **Initial distribution** $\pi \in \mathbb{R}^N$: $\pi_i = P(Z_1 = q_i)$

Generative process:

1. Draw $Z_1 \sim \pi$; emit $O_1 \sim B[Z_1, \cdot]$
2. For $t = 2, \dots, T$: draw $Z_t \sim A[Z_{t-1}, \cdot]$; emit $O_t \sim B[Z_t, \cdot]$

Hidden states $Z_{1:T}$ are *never* observed; we only see $O_{1:T}$.

HMM Structure



- The hidden states $Z_1, \dots, Z_T$ form a *Markov chain* (governed by A and π)
- Each observation O_t depends *only* on the current hidden state Z_t (governed by B)

Three Fundamental Problems

Given model $\lambda = (A, B, \pi)$ and observations $O = (O_1, \dots, O_T)$:

1. **Evaluation** — What is the probability of the observed sequence?

$$P(O \mid \lambda) = ?$$

Algorithm: Forward algorithm

Complexity: $O(N^2T)$

2. **Decoding** — What is the most likely hidden state sequence?

$$Z^* = \arg \max_{Z_{1:T}} P(Z_{1:T} \mid O, \lambda)$$

Algorithm: Viterbi algorithm

Complexity: $O(N^2T)$

3. **Learning** — Given observations only, how do we estimate λ ?

$$\hat{\lambda} = \arg \max_{\lambda} P(O \mid \lambda)$$

Algorithm: Baum–Welch (EM)

Complexity: $O(N^2T)$ per iteration

Problem 1: Evaluation – Forward Algorithm

Naïve approach: sum over all N^T state sequences — exponential in T .

Forward variable: $\alpha_t(i) = P(O_1, \dots, O_t, Z_t = q_i \mid \lambda)$

Initialization ($t = 1$):

$$\alpha_1(i) = \pi_i b_{i, O_1}$$

Recursion ($t = 2, \dots, T$):

$$\alpha_t(j) = \left[\sum_{i=1}^N \alpha_{t-1}(i) a_{ij} \right] b_{j, O_t}$$

Termination:

$$P(O \mid \lambda) = \sum_{i=1}^N \alpha_T(i)$$

The recursion reuses all intermediate computations: $O(N^2T)$ instead of $O(N^T)$.

Problem 2: Decoding – Viterbi Algorithm

Find the single best path: $Z^* = \arg \max_{Z_{1:T}} P(Z_{1:T} \mid O, \lambda)$.

Viterbi variable: $\delta_t(i) = \max_{Z_{1:t-1}} P(Z_{1:t-1}, Z_t = q_i \mid O_{1:t}, \lambda)$

Initialization:

$$\delta_1(i) = \pi_i b_{i,O_1}, \quad \psi_1(i) = 0$$

Recursion:

$$\delta_t(j) = \max_{1 \leq i \leq N} [\delta_{t-1}(i) a_{ij}] b_{j,O_t}, \quad \psi_t(j) = \arg \max_{1 \leq i \leq N} [\delta_{t-1}(i) a_{ij}]$$

Traceback: start from $Z_T^* = \arg \max_i \delta_T(i)$, then $Z_t^* = \psi_{t+1}(Z_{t+1}^*)$ backwards.

Same structure as the forward algorithm with **max** replacing **sum**.

Problem 3: Learning – Baum–Welch

Goal: find $\lambda = (A, B, \pi)$ maximising $P(O \mid \lambda)$. No closed form — use EM.

If we knew $Z_{1:T}$, we'd estimate $\hat{a}_{ij} = \frac{\#(Z_t=q_i, Z_{t+1}=q_j)}{\#(Z_t=q_i)}$, $\hat{b}_{ik} = \frac{\#(Z_t=q_i, O_t=v_k)}{\#(Z_t=q_i)}$.

Forward variable $\alpha_t(i)$

$$\alpha_t(i) = P(O_1, \dots, O_t, Z_t=q_i \mid \lambda)$$

$$\text{Init: } \alpha_1(i) = \pi_i b_{i,O_1}$$

Recurse:

$$\alpha_{t+1}(i) = \sum_j \alpha_t(j) a_{ji} b_{i,O_{t+1}}$$

Sweeps left $\rightarrow$ right.

Backward variable $\beta_t(i)$

$$\beta_t(i) = P(O_{t+1}, \dots, O_T \mid Z_t=q_i, \lambda)$$

$$\text{Init: } \beta_T(i) = 1$$

Recurse:

$$\beta_t(i) = \sum_j a_{ij} b_{j,O_{t+1}} \beta_{t+1}(j)$$

Sweeps right $\rightarrow$ left.

Problem 3: Learning – Baum–Welch (EM)

E-step:

$$\gamma_t(i) = P(Z_t = q_i \mid O, \lambda) = \frac{P(Z_t = q_i, O \mid \lambda)}{P(O \mid \lambda)} \propto \alpha_t(i) \beta_t(i)$$

$$\xi_t(i, j) = P(Z_t = q_i, Z_{t+1} = q_j \mid O, \lambda) = \frac{P(Z_t = q_i, Z_{t+1} = q_j, O \mid \lambda)}{P(O \mid \lambda)} \propto \alpha_t(i) a_{ij} b_{j, O_{t+1}} \beta_{t+1}(j)$$

M-step: re-estimate parameters:

$$\hat{\pi}_i = \gamma_1(i), \quad \hat{a}_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}, \quad \hat{b}_{ik} = \frac{\sum_{t: O_t=v_k} \gamma_t(i)}{\sum_{t=1}^T \gamma_t(i)}$$

Iterate E and M steps until convergence (guaranteed to find a local maximum of $P(O \mid \lambda)$).

Example: Part-of-Speech Tagging

Task: assign a POS tag to each word in a sentence.

“The cat sat on the mat” $\xrightarrow{\text{Viterbi}}$ Det Noun Verb Prep Det Noun

Model components

- *Hidden states:* POS tags
- *Observations:* words (vocabulary)
- *Transitions:* $P(\text{Noun} \mid \text{Det})$
- *Emissions:* $P(\text{“cat”} \mid \text{Noun})$

Mapping to the three problems

- *Evaluation:* how likely is this sentence?
- *Decoding:* what POS sequence best explains the words? (Viterbi)
- *Learning:* supervised (annotated corpora) or unsupervised (Baum–Welch)

References

-  L. R. Rabiner. A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989.
<https://www.cs.ubc.ca/~murphyk/Bayes/rabiner.pdf>
-  D. Jurafsky, J. H. Martin. *Speech and Language Processing*, 3rd ed., 2024. Chapter 17: Sequence Labeling for Parts of Speech and Named Entities.
https://web.stanford.edu/~jurafsky/slp3/ed3bookaug20_2024.pdf
-  C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. Chapter 13: Sequential Data.
<https://www.microsoft.com/en-us/research/wp-content/uploads/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf>
-  S. Brin, L. Page. *The Anatomy of a Large-Scale Hypertextual Web Search Engine*. Stanford University, 1998.
<http://infolab.stanford.edu/pub/papers/google.pdf>